

ASSIGNMENT – FINAL PROJECT DAV-5100

BILLY MUCHIPISI

NAMITA CHHANTYAL

DARLINGTON MASODZI

Disease Data Management and Analytics System

This presentation outlines the development of a comprehensive disease data management system aimed at improving public health outcomes.

We cover problem definition, database design, implementation, analytics, and modern cloud deployment strategies. The system supports tracking diseases, analyzing treatment effectiveness, and identifying risk factors to enable early intervention and informed decision-making.



Defining the Business Problem and Data Model

The goal is to enhance public health by tracking diseases, analyzing treatment effectiveness, and studying patient demographics and risk factors. Public health agencies require a reliable database system to monitor racial and geographic disparities and support operational and strategic analytics.

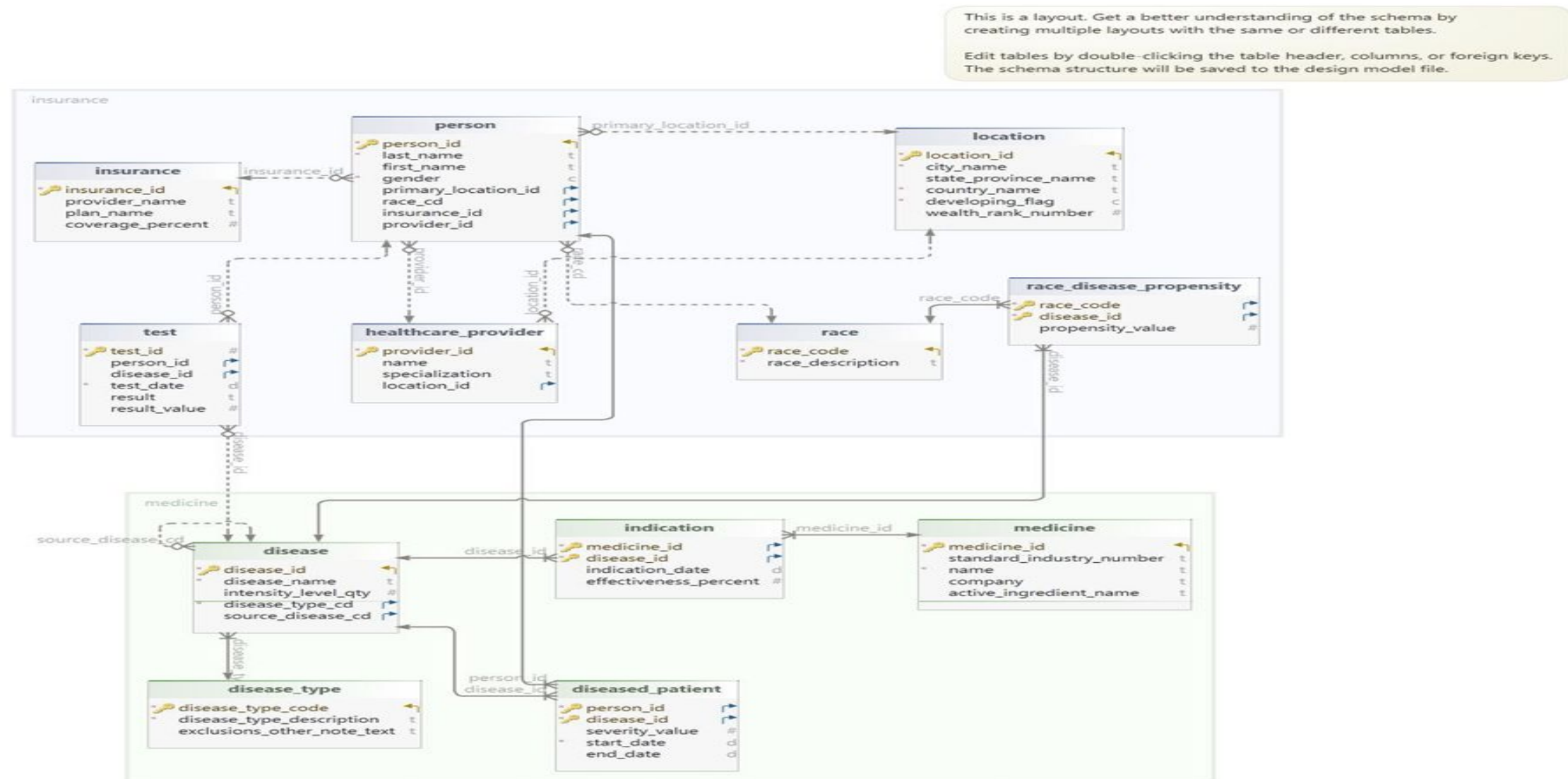
Business Goal

Improve health outcomes via disease tracking and analytics.

Data Model

Relational schema with entities for patients, diseases, tests, and locations.

ER DIAGRAM



The relational **ER diagram** and data dictionary define the schema, capturing entities like patients, diseases, tests, and locations to structure the data effectively.

DATA DICTIONARY

Table disease

Idx	Column Name	Data Type
*🔑	disease_id	serial
*	disease_name	varchar(100)
	intensity_level_qty	integer DEFAULT 1
*🔗	disease_type_cd	char(5)
🔗	source_disease_cd	integer
Indexes		
🔑	disease_pkey	Primary Key ON disease_id
Foreign Key		
🔗	disease_disease_type_cd_fkey	disease_type_cd ↗ 📄 disease_type (disease_type_code)
🔗	disease_source_disease_cd_fkey	source_disease_cd ↗ 📄 disease (disease_id)
Referring Foreign Key		
🔗	disease_source_disease_cd_fkey	disease_id ✓ 📄 disease (source_disease_cd)
🔗	diseased_patient_disease_id_fkey	disease_id ✓ 📄 diseased_patient
🔗	indication_disease_id_fkey	disease_id ✓ 📄 indication
🔗	race_disease_propensity_disease_id_fkey	disease_id ✓ 📄 race_disease_propensity
🔗	test_disease_id_fkey	disease_id ✓ 📄 test
Constraints		
	disease_intensity_level_qty_check	((intensity_level_qty >= 1) AND (intensity_level_qty <= 10))

Table diseased_patient

Idx	Column Name	Data Type
*🔗	person_id	integer
*🔗	disease_id	integer
	severity_value	integer DEFAULT 1
*	start_date	date
	end_date	date
Indexes		
🔑	diseased_patient_pkey	Primary Key ON person_id, disease_id
Foreign Key		
🔗	diseased_patient_person_id_fkey	person_id ↗ 📄 person
🔗	diseased_patient_disease_id_fkey	disease_id ↗ 📄 disease
Constraints		
	diseased_patient_severity_value_check	((severity_value >= 1) AND (severity_value <= 10))

Table test

Idx	Column Name	Data Type
*🔑	test_id	serial
🔗	person_id	integer
🔗	disease_id	integer
*	test_date	date
	result	varchar(50)
	result_value	double precision
Indexes		
🔑	test_pkey	Primary Key ON test_id
Foreign Key		
🔗	test_person_id_fkey	person_id ↗ 📄 person
🔗	test_disease_id_fkey	disease_id ↗ 📄 disease

Creating Tables

Query	Query History	Query	Query History
1	-- 1. Disease Type	41	-- 6. Healthcare Provider
2	CREATE TABLE disease_type (	42	CREATE TABLE healthcare_provider (
3	disease_type_code CHAR(5) PRIMARY KEY,	43	provider_id SERIAL PRIMARY KEY,
4	disease_type_description VARCHAR(1000) NOT NULL,	44	name VARCHAR(150),
5	exclusions_other_note_text VARCHAR(2000)	45	specialization VARCHAR(100),
6	);	46	location_id INT REFERENCES location(location_id)
7		47	);
8	-- 2. Disease	48	
9	CREATE TABLE disease (	49	-- 7. Person
10	disease_id SERIAL PRIMARY KEY,	50	CREATE TABLE person (
11	disease_name VARCHAR(100) NOT NULL,	51	person_id SERIAL PRIMARY KEY,
12	intensity_level_qty INT DEFAULT 1 CHECK (intensity_level_qty BETWEEN 1 AND 10),	52	last_name VARCHAR(50) NOT NULL,
13	disease_type_cd CHAR(5) NOT NULL REFERENCES disease_type(disease_type_code),	53	first_name VARCHAR(50),
14	source_disease_cd INT REFERENCES disease(disease_id)	54	gender CHAR(1) NOT NULL CHECK (gender IN ('M', 'F', 'U')),
15	);	55	primary_location_id INT REFERENCES location(location_id),
16		56	race_cd CHAR(5) REFERENCES race(race_code),
17	-- 3. Race	57	insurance_id INT REFERENCES insurance(insurance_id),
18	CREATE TABLE race (	58	provider_id INT REFERENCES healthcare_provider(provider_id)
19	race_code CHAR(5) PRIMARY KEY,	59	);
20	race_description VARCHAR(100) NOT NULL	60	
21	);	61	-- 8. Diseased Patient
22		62	CREATE TABLE diseased_patient (
23	-- 4. Location	63	person_id INT NOT NULL REFERENCES person(person_id),
24	CREATE TABLE location (	64	disease_id INT NOT NULL REFERENCES disease(disease_id),
25	location_id SERIAL PRIMARY KEY,	65	severity_value INT DEFAULT 1 CHECK (severity_value BETWEEN 1 AND 10),
26	city_name VARCHAR(100) NOT NULL,	66	start_date DATE NOT NULL,
27	state_province_name VARCHAR(100),	67	end_date DATE,
28	country_name VARCHAR(100) NOT NULL,	68	PRIMARY KEY (person_id, disease_id)
29	developing_flag CHAR(1) NOT NULL CHECK (developing_flag IN ('Y', 'N')),	69	);
30	wealth_rank_number INT CHECK (wealth_rank_number BETWEEN 1 AND 10)	70	
31	);	71	-- 9. Medicine
32		72	CREATE TABLE medicine (
33	-- 5. Insurance	73	medicine_id SERIAL PRIMARY KEY,
34	CREATE TABLE insurance (	74	standard_industry_number VARCHAR(25),
35	insurance_id SERIAL PRIMARY KEY,	75	name VARCHAR(250) NOT NULL,
36	provider_name VARCHAR(150),	76	company VARCHAR(150),
37	plan_name VARCHAR(150),	77	active_ingredient_name VARCHAR(150)
38	coverage_percent FLOAT CHECK (coverage_percent BETWEEN 0 AND 100)	78	);
39	);	79	
40		80	-- 10. Indication
		81	CREATE TABLE indication (
		82	medicine_id INT NOT NULL REFERENCES medicine(medicine_id),
		83	disease_id INT NOT NULL REFERENCES disease(disease_id),
		84	indication_date DATE,
		85	effectiveness_percent FLOAT CHECK (effectiveness_percent BETWEEN 0 AND 100),
		86	PRIMARY KEY (medicine_id, disease_id)
		87	);
		88	
		89	-- 11. Race Disease Propensity
		90	CREATE TABLE race_disease_propensity (
		91	race_code CHAR(5) NOT NULL REFERENCES race(race_code),
		92	disease_id INT NOT NULL REFERENCES disease(disease_id),
		93	propensity_value INT CHECK (propensity_value BETWEEN 1 AND 10),
		94	PRIMARY KEY (race_code, disease_id)
		95	);
		96	
		97	-- 12. Test
		98	CREATE TABLE test (
		99	test_id SERIAL PRIMARY KEY,
		100	person_id INT REFERENCES person(person_id),
		101	disease_id INT REFERENCES disease(disease_id),
		102	test_date DATE NOT NULL,
		103	result VARCHAR(50),
		104	result_value FLOAT
		105	);
		106	

Populating Tables

```
2  -- 1. Disease Type
3  ✓ INSERT INTO disease_type (disease_type_code, disease_type_description, exclusions_other_note_text) VALUES
4  ('INF', 'Infectious Disease', 'None'),
5  ('CHR', 'Chronic Disease', 'Exclude mental disorders'),
6  ('GEN', 'Genetic Disorder', 'Inherited conditions'),
7  ('NUR', 'Neurological Disorder', 'Exclude trauma-induced'),
8  ('CAN', 'Cancer', 'Malignant only'),
9  ('MNT', 'Mental Health Disorder', 'Psychological conditions'),
10 ('IMM', 'Immune System Disorder', 'Autoimmune types'),
11 ('CAR', 'Cardiovascular Disease', 'Heart-related conditions'),
12 ('RES', 'Respiratory Disease', 'Exclude allergy only'),
13 ('END', 'Endocrine Disorder', 'Hormonal issues');
14
15  -- 2. Disease
16  ✓ INSERT INTO disease (disease_name, intensity_level_qty, disease_type_cd, source_disease_cd) VALUES
17  ('COVID-19', 8, 'INF', NULL),
18  ('Diabetes', 5, 'CHR', NULL),
19  ('Tuberculosis', 7, 'INF', NULL),
20  ('Breast Cancer', 9, 'CAN', NULL),
21  ('Depression', 6, 'MNT', NULL),
22  ('Asthma', 5, 'RES', NULL),
23  ('Lupus', 7, 'IMM', NULL),
24  ('Heart Attack', 9, 'CAR', NULL),
25  ('Parkinson's', 6, 'NUR', NULL),
26  ('Thyroid Disorder', 4, 'END', NULL);
27
28  -- 3. Race
29  ✓ INSERT INTO race (race_code, race_description) VALUES
30  ('ASN', 'Asian'),
31  ('BLK', 'Black or African American'),
32  ('WHT', 'White'),
33  ('LAT', 'Latino/Hispanic'),
34  ('NAT', 'Native American'),
35  ('PAC', 'Pacific Islander'),
36  ('MID', 'Middle Eastern'),
37  ('MUL', 'Multiracial'),
38  ('OTH', 'Other'),
39  ('UND', 'Undisclosed');
```

Query Query History

```
41  -- 4. Location
42  ✓ INSERT INTO location (city_name, state_province_name, country_name, developing_flag, wealth_rank_number) VALUES
43  ('New York', 'NY', 'USA', 'N', 9),
44  ('Lalitpur', 'Bagmati', 'Nepal', 'Y', 5),
45  ('London', 'England', 'UK', 'N', 10),
46  ('Delhi', 'Delhi', 'India', 'Y', 6),
47  ('Tokyo', 'Tokyo', 'Japan', 'N', 10),
48  ('Lagos', 'Lagos', 'Nigeria', 'Y', 3),
49  ('Berlin', 'Berlin', 'Germany', 'N', 9),
50  ('Rio', 'RJ', 'Brazil', 'Y', 6),
51  ('Toronto', 'ON', 'Canada', 'N', 9),
52  ('Sydney', 'NSW', 'Australia', 'N', 10);
53
54
55  -- 5. Insurance
56  ✓ INSERT INTO insurance (provider_name, plan_name, coverage_percent) VALUES
57  ('Aetna', 'Silver Plan', 80),
58  ('Blue Cross', 'Gold Plan', 90),
59  ('United Health', 'Bronze Plan', 70),
60  ('Cigna', 'Platinum Plan', 95),
61  ('Humana', 'Basic Care', 75),
62  ('Kaiser', 'Family Health', 85),
63  ('Anthem', 'Essential Health', 88),
64  ('WellCare', 'WellBasic', 73),
65  ('Oscar', 'Core Plan', 78),
66  ('Molina', 'Complete Care', 92);
67
68  -- 6. Healthcare Provider
69  ✓ INSERT INTO healthcare_provider (name, specialization, location_id) VALUES
70  ('Dr. Smith', 'Internal Medicine', 1),
71  ('Dr. Sharma', 'Pulmonologist', 2),
72  ('Dr. Ahmed', 'Cardiologist', 3),
73  ('Dr. Lee', 'Neurologist', 4),
74  ('Dr. Mehta', 'Oncologist', 5),
75  ('Dr. Chen', 'Endocrinologist', 6),
76  ('Dr. Patel', 'General Practitioner', 7),
77  ('Dr. Gomez', 'Psychiatrist', 8),
78  ('Dr. Kim', 'Immunologist', 9),
79  ('Dr. Singh', 'Family Medicine', 10);
80
```

```
121  -- 10. Diseased Patient
122  ✓ INSERT INTO diseased_patient (person_id, disease_id, severity_value, start_date, end_date) VALUES
123  (1, 1, 7, '2020-05-15', '2020-06-30'),
124  (2, 2, 6, '2019-03-10', NULL),
125  (3, 3, 8, '2021-11-05', NULL),
126  (4, 4, 9, '2022-02-01', NULL),
127  (5, 5, 5, '2021-01-20', NULL),
128  (6, 6, 6, '2020-12-10', NULL),
129  (7, 7, 7, '2021-03-15', NULL),
130  (8, 8, 8, '2022-07-07', NULL),
131  (9, 9, 6, '2023-05-01', NULL),
132  (10, 10, 5, '2021-09-25', NULL);
133
134  -- 11. Race Disease Propensity
135  ✓ INSERT INTO race_disease_propensity (race_code, disease_id, propensity_value) VALUES
136  ('ASN', 2, 7),
137  ('BLK', 2, 8),
138  ('WHT', 1, 5),
139  ('LAT', 5, 6),
140  ('NAT', 4, 7),
141  ('PAC', 3, 6),
142  ('MID', 8, 8),
143  ('MUL', 6, 7),
144  ('OTH', 7, 5),
145  ('UND', 9, 6);
146
147  -- 12. Test
148  ✓ INSERT INTO test (person_id, disease_id, test_date, result, result_value) VALUES
149  (1, 1, '2020-05-10', 'Positive', 1.0),
150  (2, 2, '2021-03-05', 'Negative', 0.0),
151  (3, 3, '2021-11-04', 'Positive', 1.0),
152  (4, 4, '2022-02-10', 'Positive', 1.0),
153  (5, 5, '2021-01-25', 'Negative', 0.0),
154  (6, 6, '2020-12-15', 'Positive', 1.0),
155  (7, 7, '2021-03-20', 'Negative', 0.0),
156  (8, 8, '2022-07-10', 'Positive', 1.0),
157  (9, 9, '2023-05-03', 'Negative', 0.0),
158  (10, 10, '2021-09-27', 'Positive', 1.0);
159
```

Query Query History

```
81  -- 7. Person
82  ✓ INSERT INTO person (last_name, first_name, gender, primary_location_id, race_cd, insurance_id, provider_id) VALUES
83  ('Lee', 'Min', 'F', 1, 'ASN', 1, 1),
84  ('Jackson', 'Tina', 'F', 1, 'BLK', 2, 1),
85  ('Bhandari', 'Raj', 'M', 2, 'ASN', 3, 2),
86  ('Smith', 'John', 'M', 3, 'WHT', 4, 3),
87  ('Khan', 'Sara', 'F', 4, 'MID', 5, 4),
88  ('Perez', 'Luis', 'M', 5, 'LAT', 6, 5),
89  ('Whitecloud', 'Maya', 'F', 6, 'NAT', 7, 6),
90  ('Park', 'Jin', 'M', 7, 'PAC', 8, 7),
91  ('Nguyen', 'Linh', 'F', 8, 'ASN', 9, 8),
92  ('Brown', 'Alicia', 'F', 9, 'BLK', 10, 9);
93
94  -- 8. Medicine
95  ✓ INSERT INTO medicine (standard_industry_number, name, company, active_ingredient_name) VALUES
96  ('SIN123', 'Remdesivir', 'Gilead', 'GS-5734'),
97  ('SIN456', 'Metformin', 'Sun Pharma', 'Metformin Hydrochloride'),
98  ('SIN789', 'Aspirin', 'Bayer', 'Acetylsalicylic Acid'),
99  ('SIN101', 'Lipitor', 'Pfizer', 'Atorvastatin'),
100 ('SIN202', 'Prozac', 'Eli Lilly', 'Fluoxetine'),
101 ('SIN303', 'Prednisone', 'Teva', 'Prednisone'),
102 ('SIN404', 'Lisinopril', 'Novartis', 'Lisinopril'),
103 ('SIN505', 'Tamoxifen', 'AstraZeneca', 'Tamoxifen Citrate'),
104 ('SIN606', 'Levothyroxine', 'AbbVie', 'Levothyroxine'),
105 ('SIN707', 'Insulin', 'Novo Nordisk', 'Insulin Human');
106
107  -- 9. Indication
108  ✓ INSERT INTO indication (medicine_id, disease_id, indication_date, effectiveness_percent) VALUES
109  (1, 1, '2020-06-01', 65.0),
110  (2, 2, '2018-01-15', 78.5),
111  (3, 8, '2021-02-01', 84.2),
112  (4, 8, '2020-03-21', 89.0),
113  (5, 5, '2022-05-10', 60.0),
114  (6, 7, '2021-07-14', 74.6),
115  (7, 8, '2019-09-30', 77.5),
116  (8, 4, '2021-11-01', 81.1),
117  (9, 10, '2022-03-10', 68.0),
118  (10, 2, '2017-10-05', 88.9);
119
120
```


Operational Queries and Data Manipulation

Query

Query History

```

1  -- 1. List all patients and the diseases they have
2  SELECT p.first_name, p.last_name, d.disease_name, dp.severity_value
3  FROM diseased_patient dp
4  JOIN person p ON dp.person_id = p.person_id
5  JOIN disease d ON dp.disease_id = d.disease_id;
6

```

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑️

📦

📥

📈

SQL

	first_name character varying (50) 🔒	last_name character varying (50) 🔒	disease_name character varying (100) 🔒	severity_value integer 🔒
1	Min	Lee	COVID-19	7
2	Tina	Jackson	Diabetes	6
3	Raj	Bhandari	Tuberculosis	8
4	John	Smith	Breast Cancer	9
5	Sara	Khan	Depression	5
6	Luis	Perez	Asthma	6
7	Maya	Whitecloud	Lupus	7
8	Jin	Park	Heart Attack	8
9	Linh	Nguyen	Parkinson's	6
10	Alicia	Brown	Thyroid Disorder	5

```
7 -- 2. Find the most common diseases by race
8 SELECT r.race_description, d.disease_name, COUNT(*) AS case_count
9 FROM diseased_patient dp
10 JOIN person p ON dp.person_id = p.person_id
11 JOIN disease d ON dp.disease_id = d.disease_id
12 JOIN race r ON p.race_cd = r.race_code
13 GROUP BY r.race_description, d.disease_name
14 ORDER BY case_count DESC;
```

Data Output Messages Notifications

	race_description character varying (100)	disease_name character varying (100)	case_count bigint
1	Native American	Lupus	1
2	Asian	COVID-19	1
3	White	Breast Cancer	1
4	Black or African American	Diabetes	1
5	Middle Eastern	Depression	1
6	Asian	Tuberculosis	1
7	Asian	Parkinson's	1
8	Pacific Islander	Heart Attack	1
9	Black or African American	Thyroid Disorder	1
10	Latino/Hispanic	Asthma	1

```

16 -- 3. Get medicine effectiveness for each disease
17 SELECT d.disease_name, m.name AS medicine_name, i.effectiveness_percent
18 FROM indication i
19 JOIN disease d ON i.disease_id = d.disease_id
20 JOIN medicine m ON i.medicine_id = m.medicine_id;
21

```

Data Output Messages Notifications

	disease_name character varying (100)	medicine_name character varying (250)	effectiveness_percent double precision
1	COVID-19	Remdesivir	65
2	Diabetes	Metformin	78.5
3	Heart Attack	Aspirin	84.2
4	Heart Attack	Lipitor	89
5	Depression	Prozac	60
6	Lupus	Prednisone	74.6
7	Heart Attack	Lisinopril	77.5
8	Breast Cancer	Tamoxifen	81.1
9	Thyroid Disorder	Levothyroxine	68
10	Diabetes	Insulin	88.9

```
-- 4. Find patients who tested positive for any disease
SELECT p.first_name, p.last_name, d.disease_name, t.test_date
FROM test t
JOIN person p ON t.person_id = p.person_id
JOIN disease d ON t.disease_id = d.disease_id
WHERE t.result = 'Positive';
```

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

🗑️

📦

⬇️

📈

SQL

	first_name character varying (50) 🔒	last_name character varying (50) 🔒	disease_name character varying (100) 🔒	test_date date 🔒
1	Min	Lee	COVID-19	2020-05-10
2	Raj	Bhandari	Tuberculosis	2021-11-04
3	John	Smith	Breast Cancer	2022-02-10
4	Luis	Perez	Asthma	2020-12-15
5	Jin	Park	Heart Attack	2022-07-10
6	Alicia	Brown	Thyroid Disorder	2021-09-27

DML OPERATIONS

```
Query Query History
1 -- Insert a new test for an existing patient
2 v INSERT INTO test (person_id, disease_id, test_date, result, result_value)
3 VALUES (1, 2, '2024-01-01', 'Negative', 0.0);
4
Data Output Messages Notifications
INSERT 0 1

Query returned successfully in 104 msec.
```

```
10 -- Try deleting a medicine (will fail if it is referenced in indication table)
11 v DELETE FROM medicine
12 WHERE medicine_id = 1;
13
Data Output Messages Notifications
ERROR: Key (medicine_id)=(1) is still referenced from table "indication".update or delete on table "medicine" violates foreign key constraint "indication_medicine_id_fkey" on table "indication"

ERROR: update or delete on table "medicine" violates foreign key constraint "indication_medicine_id_fkey" on table "indication"
SQL state: 23503
Detail: Key (medicine_id)=(1) is still referenced from table "indication".
```

```
5 -- Update severity of a disease case
6 v UPDATE diseased_patient
7 SET severity_value = 9
8 WHERE person_id = 2 AND disease_id = 2;
9
Data Output Messages Notifications
UPDATE 1

Query returned successfully in 89 msec.
```

```
14 -- Delete test data for a patient (if allowed)
15 v DELETE FROM test
16 WHERE person_id = 3;
17 |
Data Output Messages Notifications
DELETE 1

Query returned successfully in 86 msec.
```

Dimensional Modeling with Star Schema

A star schema dimensional model is created to support analytical queries and business intelligence. Dimension tables capture attributes like patient demographics and disease details.

Slowly Changing Dimension Type 2 (SCD2) is implemented to track historical changes in dimension data, preserving data accuracy over time.

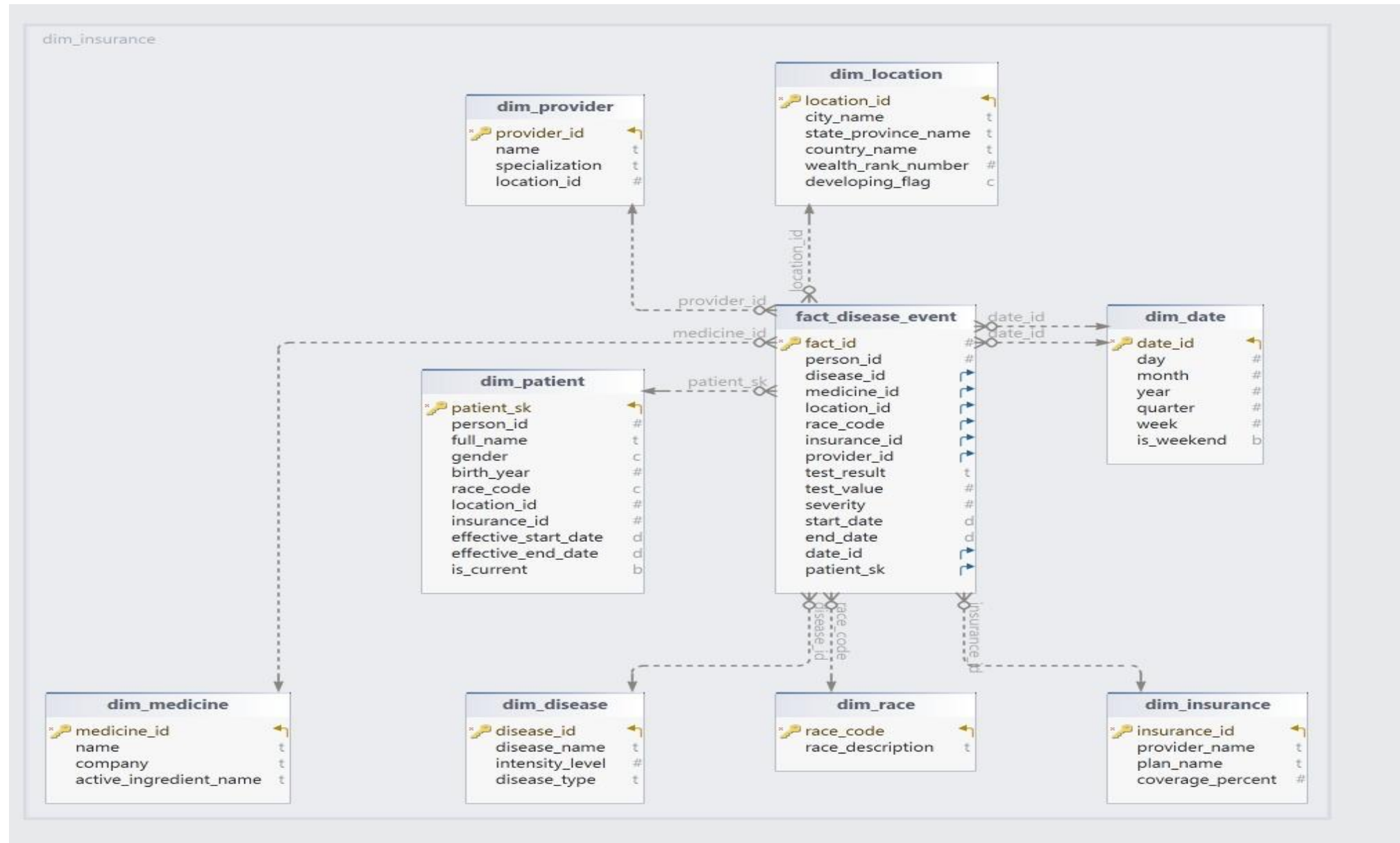
Dimension Tables

Store descriptive attributes for analysis.

SCD Type 2

Maintains historical versions of dimension records.

Dimensional Modeling- Disease DW



Creating and Loading Dimensions

```
1
2 -- Create schema for Data Warehouse
3 CREATE SCHEMA IF NOT EXISTS disease_dwh;
4 SET search_path TO disease_dwh;
5
6 -- 1. dim_patient
7 CREATE TABLE dim_patient (
8     patient_id SERIAL PRIMARY KEY,
9     person_id INT,
10    full_name VARCHAR(100),
11    gender CHAR(1),
12    birth_year INT,
13    race_code CHAR(5),
14    location_id INT
15 );
16
17 -- 2. dim_disease
18 CREATE TABLE dim_disease (
19     disease_id INT PRIMARY KEY,
20     disease_name VARCHAR(100),
21     intensity_level INT,
22     disease_type VARCHAR(100)
23 );
24
25 -- 3. dim_medicine
26 CREATE TABLE dim_medicine (
27     medicine_id INT PRIMARY KEY,
28     name VARCHAR(250),
29     company VARCHAR(150),
30     active_ingredient_name VARCHAR(150)
31 );
32
```

Query	Query History
33	-- 4. dim_location
34	CREATE TABLE dim_location (
35	location_id INT PRIMARY KEY,
36	city_name VARCHAR(100),
37	state_province_name VARCHAR(100),
38	country_name VARCHAR(100),
39	wealth_rank_number INT,
40	developing_flag CHAR(1)
41	);
43	-- 5. dim_race
44	CREATE TABLE dim_race (
45	race_code CHAR(5) PRIMARY KEY,
46	race_description VARCHAR(100)
47	);
48	
49	-- 6. dim_insurance
50	CREATE TABLE dim_insurance (
51	insurance_id INT PRIMARY KEY,
52	provider_name VARCHAR(150),
53	plan_name VARCHAR(150),
54	coverage_percent FLOAT
55	);
56	
57	-- 7. dim_provider
58	CREATE TABLE dim_provider (
59	provider_id INT PRIMARY KEY,
60	name VARCHAR(150),
61	specialization VARCHAR(100),
62	location_id INT
63	);
64	

```
65 -- 8. dim_date
66 CREATE TABLE disease_dwh.dim_date (
67     date_id DATE PRIMARY KEY,
68     day INT,
69     month INT,
70     year INT,
71     quarter INT,
72     week INT,
73     is_weekend BOOLEAN
74 );
75
76 -- 9. fact_disease_event
77 CREATE TABLE disease_dwh.fact_disease_event (
78     fact_id SERIAL PRIMARY KEY,
79     person_id INT,
80     disease_id INT,
81     medicine_id INT,
82     location_id INT,
83     race_code CHAR(5),
84     insurance_id INT,
85     provider_id INT,
86     test_result VARCHAR(50),
87     test_value FLOAT,
88     severity INT,
89     start_date DATE,
90     end_date DATE,
91     date_id DATE REFERENCES disease_dwh.dim_date(date_id) -- FK to dim_date
92 );
93
```

```
1 -- ETL Script: Populate Dimension Tables
2
3
4 -- 1. dim_patient
5 INSERT INTO disease_dwh.dim_patient (person_id, full_name, gender, birth_year, race_code, location_id)
6 SELECT
7     person_id,
8     CONCAT(first_name, ' ', last_name),
9     gender,
10    EXTRACT(YEAR FROM CURRENT_DATE) - 30, -- Assume everyone is 30 years old for mock data
11    race_cd,
12    primary_location_id
13 FROM person;
14
15 -- 2. dim_disease
16 INSERT INTO disease_dwh.dim_disease (disease_id, disease_name, intensity_level, disease_type)
17 SELECT
18     d.disease_id,
19     d.disease_name,
20     d.intensity_level_qty,
21     dt.disease_type_description
22 FROM disease d
23 JOIN disease_type dt ON d.disease_type_cd = dt.disease_type_code;
24
25 -- 3. dim_medicine
26 INSERT INTO disease_dwh.dim_medicine (medicine_id, name, company, active_ingredient_name)
27 SELECT medicine_id, name, company, active_ingredient_name
28 FROM medicine;
29
30 -- 4. dim_location
31 INSERT INTO disease_dwh.dim_location (location_id, city_name, state_province_name, country_name, wealth_rank_number, developing_flag)
32 SELECT location_id, city_name, state_province_name, country_name, wealth_rank_number, developing_flag
33 FROM location;
34
35 -- 5. dim_race
36 INSERT INTO disease_dwh.dim_race (race_code, race_description)
37 SELECT race_code, race_description
38 FROM race;
39
```

Creating and Loading Dimensions (cont)

```
40 -- 6. dim_insurance
41 ✓ INSERT INTO disease_dwh.dim_insurance (insurance_id, provider_name, plan_name, coverage_percent)
42 SELECT insurance_id, provider_name, plan_name, coverage_percent
43 FROM insurance;
44
45 -- 7. dim_provider
46 ✓ INSERT INTO disease_dwh.dim_provider (provider_id, name, specialization, location_id)
47 SELECT provider_id, name, specialization, location_id
48 FROM healthcare_provider;
49
50 -- Generate full date range in dim_date (2020-01-01 to 2025-12-31)
51 -- Extend dim_date to start from 2015
52 ✓ INSERT INTO disease_dwh.dim_date (date_id, day, month, year, quarter, week, is_weekend)
53 SELECT
54     d::DATE,
55     EXTRACT(DAY FROM d),
56     EXTRACT(MONTH FROM d),
57     EXTRACT(YEAR FROM d),
58     EXTRACT(QUARTER FROM d),
59     EXTRACT(WEEK FROM d),
60     CASE WHEN EXTRACT(DOW FROM d) IN (0, 6) THEN TRUE ELSE FALSE END
61 FROM generate_series('2015-01-01'::DATE, '2019-12-31'::DATE, '1 day'::INTERVAL) d;
62
63
64 SELECT COUNT(*) FROM disease_dwh.dim_date;
65
```

```
66 -- 8. fact_disease_event
67 ✓ INSERT INTO disease_dwh.fact_disease_event (
68     person_id, disease_id, medicine_id, location_id, race_code,
69     insurance_id, provider_id, test_result, test_value, severity,
70     start_date, end_date, date_id
71 )
72 SELECT
73     p.person_id,
74     dp.disease_id,
75     i.medicine_id,
76     p.primary_location_id,
77     p.race_cd,
78     p.insurance_id,
79     p.provider_id,
80     t.result,
81     t.result_value,
82     dp.severity_value,
83     dp.start_date,
84     dp.end_date,
85     dp.start_date
86 FROM diseased_patient dp
87 JOIN person p ON dp.person_id = p.person_id
88 LEFT JOIN test t ON t.person_id = dp.person_id AND t.disease_id = dp.disease_id
89 LEFT JOIN indication i ON i.disease_id = dp.disease_id;
90
```

SCD -Type 2

```

82 DROP TABLE IF EXISTS disease_dwh.dim_patient;
83
84 CREATE TABLE disease_dwh.dim_patient (
85     patient_sk SERIAL PRIMARY KEY,           -- Surrogate key
86     person_id INT,                           -- Business key
87     full_name VARCHAR(100),
88     gender CHAR(1),
89     birth_year INT,
90     race_code CHAR(5),
91     location_id INT,
92     insurance_id INT,
93     effective_start_date DATE,               -- SCD2 tracking
94     effective_end_date DATE,
95     is_current BOOLEAN
96 );
97
98 SELECT * FROM disease_dwh.dim_patient;

```

Data Output Messages Notifications












patient_sk	person_id	full_name	gender	birth_year	race_code	location_id	insurance_id	effective_start_date	effective_end_date	is_current
[PK] integer	integer	character varying (100)	character (1)	integer	character (5)	integer	integer	date	date	boolean

```
5 ▾ INSERT INTO disease_dwh.dim_patient (person_id, full_name, gender, birth_year, race_code, location_id)
6 SELECT
7     person_id,
8     CONCAT(first_name, ' ', last_name),
9     gender,
10    EXTRACT(YEAR FROM CURRENT_DATE) - 30, -- Assume everyone is 30 years old for mock data
11    race_cd,
12    primary_location_id
13 FROM person;
14
15 Select * from disease_dwh.dim_patient;
16
```

[Data Output](#) [Messages](#) [Notifications](#)

Showing rows: 1 to 10 Page No: 1

	patient_sk [PK] integer	person_id integer	full_name character varying (100)	gender character (1)	birth_year integer	race_code character (5)	location_id integer	insurance_id integer	effective_start_date date	effective_end_date date	is_current boolean
1	1	1	Min Lee	F	1995	ASN	1	[null]	[null]	[null]	[null]
2	2	2	Tina Jackson	F	1995	BLK	1	[null]	[null]	[null]	[null]
3	3	3	Raj Bhandari	M	1995	ASN	2	[null]	[null]	[null]	[null]
4	4	4	John Smith	M	1995	WHT	3	[null]	[null]	[null]	[null]
5	5	5	Sara Khan	F	1995	MID	4	[null]	[null]	[null]	[null]
6	6	6	Luis Perez	M	1995	LAT	5	[null]	[null]	[null]	[null]
7	7	7	Maya Whitecloud	F	1995	NAT	6	[null]	[null]	[null]	[null]
8	8	8	Jin Park	M	1995	PAC	7	[null]	[null]	[null]	[null]
9	9	9	Linh Nguyen	F	1995	ASN	8	[null]	[null]	[null]	[null]
10	10	10	Alicia Brown	F	1995	BLK	9	[null]	[null]	[null]	[null]

```
23 INSERT INTO disease_dwh.dim_patient (
24     person_id, full_name, gender, birth_year, race_code, location_id,
25     insurance_id, effective_start_date, effective_end_date, is_current
26 )
27 VALUES (
28     3, 'Raj Bhandari', 'M', 1995, 'ASN', 5, 5,
29     CURRENT_DATE, NULL, TRUE
30 );
31
32 Select * from disease_dwh.dim_patient;
33
```

Data Output Messages Notifications

Showing rows: 1 to 11 Page No: 1

	patient_sk [PK] integer	person_id integer	full_name character varying (100)	gender character (1)	birth_year integer	race_code character (5)	location_id integer	insurance_id integer	effective_start_date date	effective_end_date date	is_current boolean
1	1	1	Min Lee	F	1995	ASN	1	[null]	[null]	[null]	[null]
2	2	2	Tina Jackson	F	1995	BLK	1	[null]	[null]	[null]	[null]
3	4	4	John Smith	M	1995	WHT	3	[null]	[null]	[null]	[null]
4	5	5	Sara Khan	F	1995	MID	4	[null]	[null]	[null]	[null]
5	6	6	Luis Perez	M	1995	LAT	5	[null]	[null]	[null]	[null]
6	7	7	Maya Whitecloud	F	1995	NAT	6	[null]	[null]	[null]	[null]
7	8	8	Jin Park	M	1995	PAC	7	[null]	[null]	[null]	[null]
8	9	9	Linh Nguyen	F	1995	ASN	8	[null]	[null]	[null]	[null]
9	10	10	Alicia Brown	F	1995	BLK	9	[null]	[null]	[null]	[null]
10	3	3	Raj Bhandari	M	1995	ASN	2	[null]	[null]	2025-05-01	false
11	11	3	Raj Bhandari	M	1995	ASN	5	5	2025-05-02	[null]	true

Analytical Queries on Star

```
1  -- Sample Analytical Queries (Star Schema)
2  --- 1. Top 5 most common diseases by Case Count
3  ✓ SELECT d.disease_name, COUNT(*) AS case_count
4  FROM disease_dwh.fact_disease_event f
5  JOIN disease_dwh.dim_disease d ON f.disease_id = d.disease_id
6  GROUP BY d.disease_name
7  ORDER BY case_count DESC
8  LIMIT 5;
```

Data Output Messages Notifications

SQL

	disease_name character varying (100)	case_count bigint
1	Heart Attack	3
2	Diabetes	2
3	Parkinson's	1
4	Tuberculosis	1
5	Breast Cancer	1

```
18 ---3. Positive Test Rates by Race
19 ✓ SELECT r.race_description, COUNT(*) FILTER (WHERE f.test_result = 'Positive')::float / COUNT(*) AS positive_rate
20 FROM disease_dwh.fact_disease_event f
21 JOIN disease_dwh.dim_race r ON f.race_code = r.race_code
22 GROUP BY r.race_description
23 ORDER BY positive_rate DESC;
```

Data Output Messages Notifications

SQL

	race_description character varying (100)	positive_rate double precision
1	Latino/Hispanic	1
2	Pacific Islander	1
3	White	1
4	Asian	0.3333333333333333
5	Black or African American	0.3333333333333333
6	Middle Eastern	0
7	Native American	0

```
24
25 ---4. Disease Count by Insurance plan
26 ✓ SELECT i.plan_name, COUNT(*) AS total_cases
27 FROM disease_dwh.fact_disease_event f
28 JOIN disease_dwh.dim_insurance i ON f.insurance_id = i.insurance_id
29 GROUP BY i.plan_name
30 ORDER BY total_cases DESC;
```

Data Output Messages Notifications

SQL

	plan_name character varying (150)	total_cases bigint
1	WellBasic	3
2	Gold Plan	2
3	Core Plan	1
4	Basic Care	1
5	Family Health	1
6	Platinum Plan	1
7	Complete Care	1
8	Essential Health	1
9	Bronze Plan	1
10	Silver Plan	1

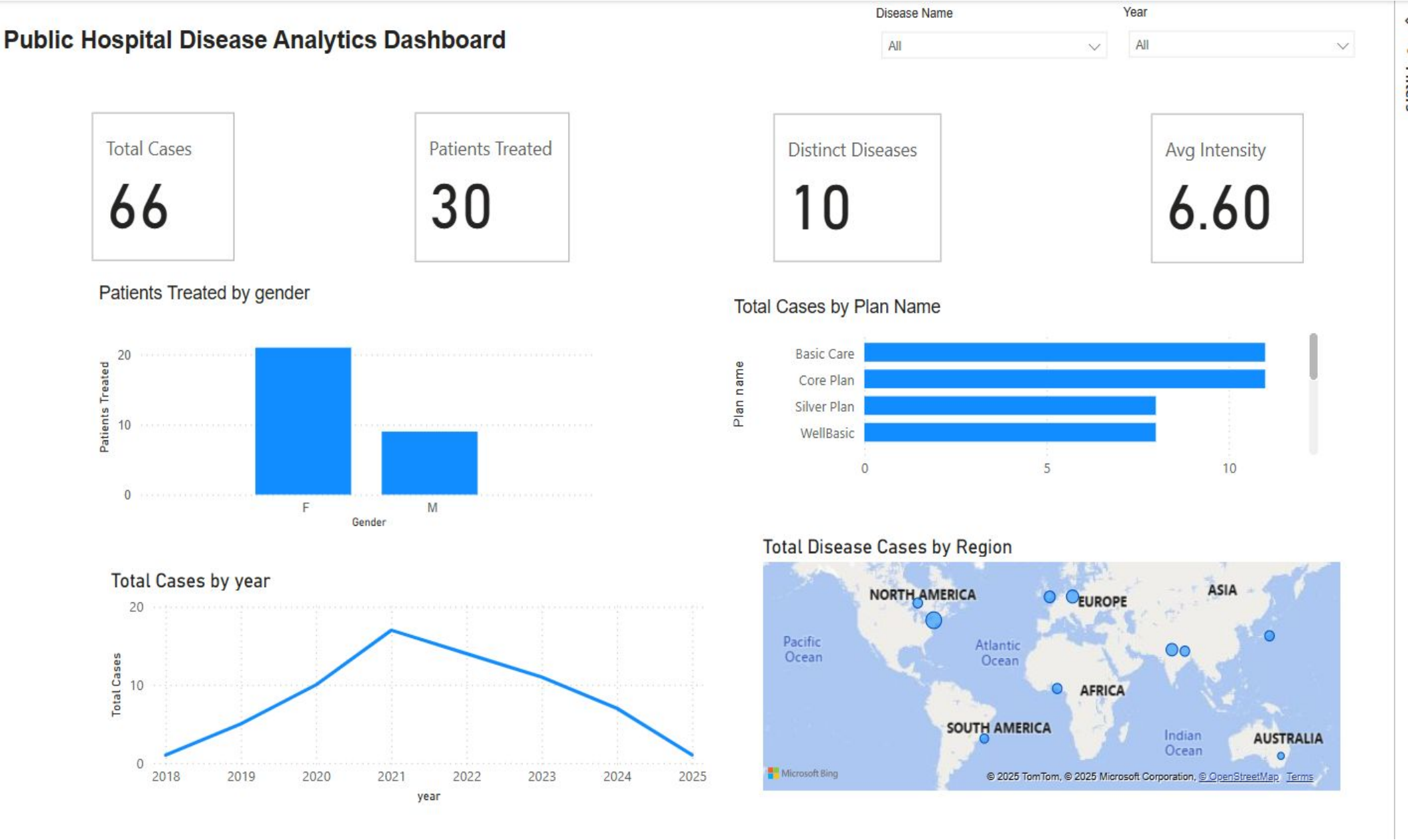
```
31
32 ---5. Monthly Case Trends
33 ✓ SELECT dd.year, dd.month, COUNT(*) AS monthly_cases
34 FROM disease_dwh.fact_disease_event f
35 JOIN disease_dwh.dim_date dd ON f.date_id = dd.date_id
36 GROUP BY dd.year, dd.month
37 ORDER BY dd.year, dd.month;
```

Data Output Messages Notifications

SQL

	year integer	month integer	monthly_cases bigint
1	2019	3	2
2	2020	5	1
3	2020	12	1
4	2021	1	1
5	2021	3	1
6	2021	9	1
7	2021	11	1
8	2022	2	1
9	2022	7	3
10	2023	5	1

POWER BI DASHBOARD



NoSQL Database Comparison

Comparing relational PostgreSQL with NoSQL alternatives reveals trade-offs. MongoDB stores embedded documents, simplifying individual patient data retrieval but risking duplication and complex analytics challenges.

Neo4j graph database excels at modeling relationships like contact tracing and disease transmission but is less suited for tabular analytics and requires specialized query languages.

MongoDB

- Embedded documents for patient-centric data
- Flexible schema, real-time applications
- Challenges with data duplication and analytics

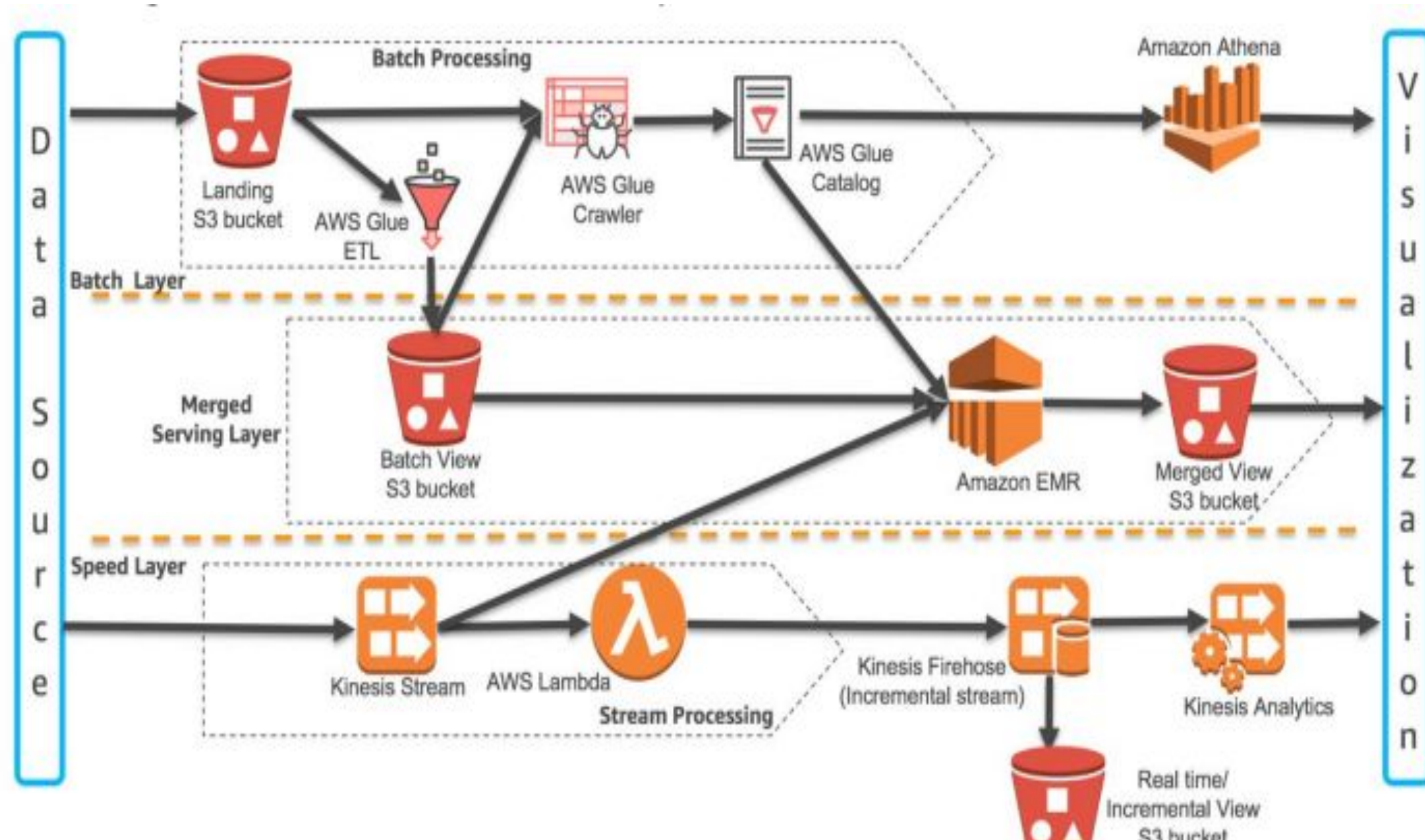
Neo4j

- Graph model for relationships and contact tracing
- Pattern matching with Cypher query language
- Limited BI tool support

```
(:Person {name: 'Raj'})  
-[:HAS_DISEASE {severity: 6}]-> (:Disease {name: 'Diabetes'})  
-[:LIVES_IN]-> (:Location {city: 'Kathmandu'})  
-[:COVERED_BY]-> (:Insurance {plan_name: 'Gold Plan'})  
-[:VISITED]-> (:Provider {name: 'Dr. Sharma'})  
-[:TESTED_FOR]-> (:Test {result: 'Positive'})
```

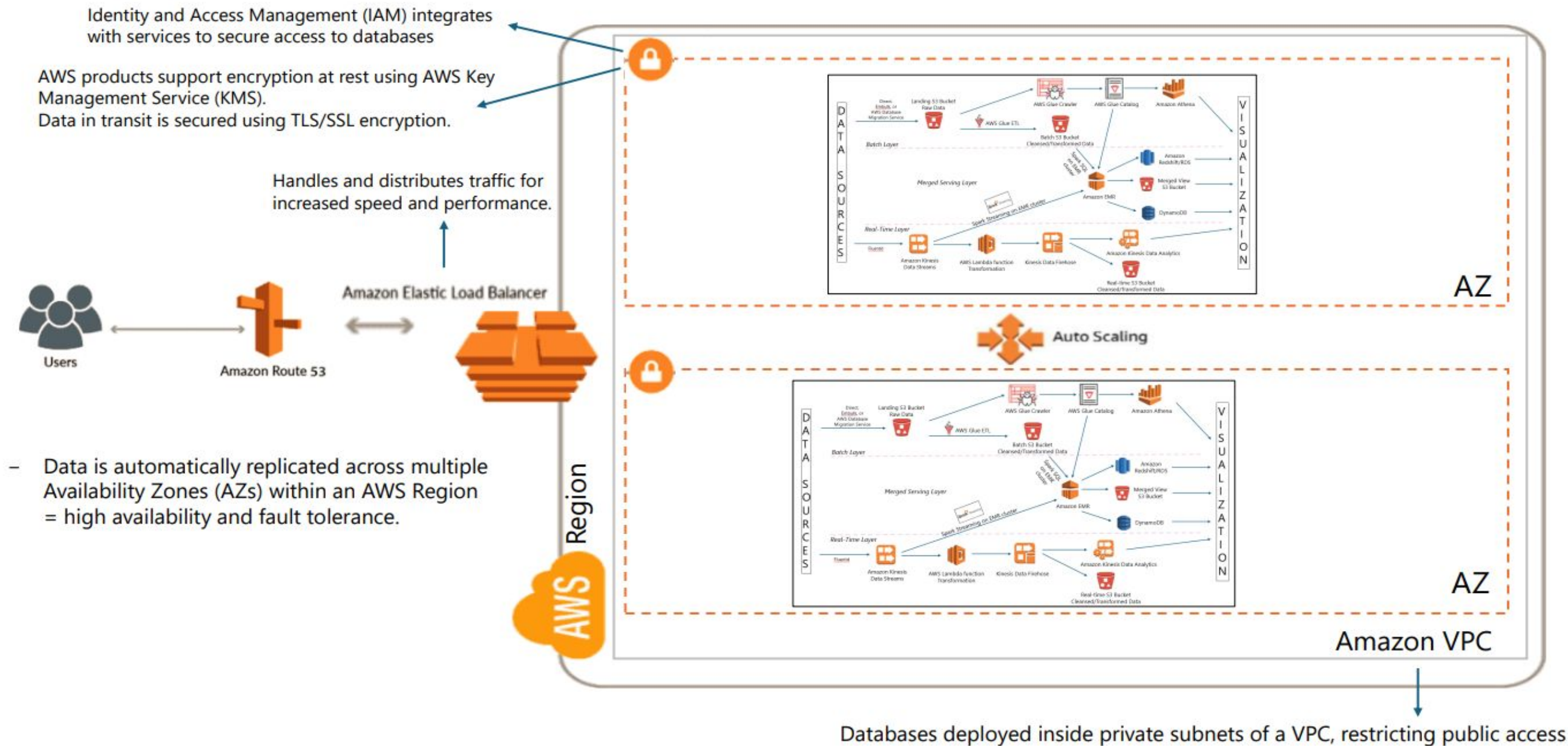
```
{  
  "person_id": 1,  
  "name": "Min Lee",  
  "gender": "F",  
  "location": {  
    "city": "New York",  
    "country": "USA"  
  },  
  "insurance": {  
    "provider_name": "Aetna",  
    "plan_name": "Silver Plan"  
  },  
  "diseases": [  
    {  
      "disease_name": "COVID-19",  
      "severity": 7,  
      "start_date": "2020-05-15",  
      "test": {  
        "result": "Positive",  
        "value": 1.0,  
        "test_date": "2020-05-10"  
      }  
    }  
  ]  
}
```

AWS Lambda Architecture for Data Processing



AWS Lambda Architecture for Data (cont)

Processing



Snowflake Advantages for Healthcare Analytics

Snowflake offers a cloud data warehouse with automatic performance tuning, elastic scalability, and separation of storage and compute. It supports concurrent dashboards and real-time data integration, ideal for large-scale healthcare analytics.

This platform complements PostgreSQL by handling complex analytical workloads efficiently, enabling secure data sharing and real-time updates for disease trend monitoring.

Performance & Scalability

Automatic tuning and consistent query speed at scale.

Real-Time & Concurrent Access

Supports simultaneous users and live data ingestion.

Conclusion

- The Disease Data Management and Analytics System enhances public health outcomes by leveraging a robust relational database model and a star schema for analytical insights.
- Incorporates Slowly Changing Dimension Type 2 (SCD2) to maintain historical data accuracy.
- Uses AWS Lambda architecture to facilitate scalable, real-time data processing.
- Integrates Snowflake for efficient handling of large-scale healthcare data with performance optimization.
- Compares relational databases with NoSQL alternatives to balance structured data management and flexible, real-time analytics.
- Supports informed decision-making for public health agencies through comprehensive data management and analytics.